

Maestría en Explotación de Datos y Descubrimiento del Conocimiento

Predicción consultas frecuentes en motores de búsqueda para el diseño de políticas de admisión en caching de resultados

Trabajo de Especialización

Autor: Delvechio Pablo Tomás



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires
Argentina
Septiembre de 2023

Resumen

La optimización de procesos en motores de búsqueda webs es central para lograr respuestas a las necesidades de usuarios en tiempos razonables. Entre los esquemas mas utilizados para optimizar parte de estos procesos están el uso de diferentes tipos de memorias de acceso rápido o memorias caché. El rendimiento de estas estructuras varia mucho en función de muchos parámetros, uno de los cuales son las políticas de reemplazo / admisión de elementos. Existen diversos tipos de políticas, pero desde hace unos años están apareciendo enfoques que incorporan algoritmos de aprendizaje automático. Esta incorporación abre un nuevo conjunto de pruebas y experimentos por hacer que pueden redundar en beneficios en el uso de estas estructuras.

Sin embargo, en la literatura se observa que las estrategias utilizadas no siempre están acordes a los últimos avances en las ciencias de datos. A veces son modelos antiguos o “clasicos”, otra veces modelos con configuración por defecto de las librerías y a veces no se reportan resultados del proceso clasificatorio sino en términos del problema original (eficiencia de una estructura de caché). Este escenario provee un espacio de vacancia para realizar estudios de ciencias de datos sobre *datasets* que abordan este problema.

En el presente trabajo se aborda el problema de clasificar consultas a motores de búsquedas que serán frecuentes en el futuro o no, a partir de bases de datos con características de las mismas. Se realiza un análisis exploratorio de datos para detectar patrones de comportamiento en el conjunto de datos, se estudia la distribución de clases y se definen recortes del *dataset* original con fines de hacer las pruebas factibles en el tiempo disponible. Se plantean experimentos con algoritmos de clasificación ampliamente estudiados por la academia y utilizados en la industria de datos, planteando una versión base con hiperparámetros por defecto y otra versión optimizada con una estrategia sencilla de búsqueda de hiperparámetros. Se realizaron experimentos sobre todos los *datasets* contruidos y se analizan los resultados. Los resultados mostraron es que optimizar el modelo de clasificación tiene efectos positivos aunque moderados en el proceso predictivo, que una búsqueda de hiperparámetros de corto alcance no tiene grandes efectos y que el crecimiento del tamaño del *dataset* impacta en la calidad del modelo clasificador.

1. Introducción

La masificación de la información digital ha generado en la última década colecciones de datos de gran tamaño. Un emergente de esta situación es el enfoque basado en los datos masivos (también conocido como *big data*). Sin embargo, previo al surgimiento del concepto, había sistemas que ya se enfrentaban a estas problemáticas de cantidad de datos difíciles o imposibles de manejar que necesitaban de estrategias escalables. Una de esas áreas es la de los motores de búsquedas web (Web Search Engines o SE).

La problemática de los motores de búsqueda web puede resumirse en: “Grupos de usuarios exigen al SE respuestas de calidad a sus consultas en el menor tiempo posible sobre una colección de documentos de escala mundial (la web)”. Este escenario se vuelve mas complejo a cada instante: Nuevos usuarios se incorporan al uso de la web constantemente; cada vez mayor es la cantidad de documentos sobre los que se realizan las búsquedas; y menores son los tiempos de respuesta que se le exigen al SE, con muy poco espacio para “negociar” la calidad de los mismos. Es razonable entonces que las investigaciones sobre la escalabilidad de motores de búsqueda web tengan una presencia permanente en la academia [3].

Una de las estrategias mas utilizadas en las ciencias de la computación para mejorar la eficiencia de los sistemas (sean estos SE u de otro tipo) son las técnicas de *caching*. Estos algoritmos permiten almacenar en memoria el resultado de un procesamiento para ser reutilizado posteriormente sin tener que ser re-computado. Diversas ventajas se obtienen al emplear una memoria caché [14]:

- Reduce la carga de trabajo en los servidores
- Reduce la utilización de ancho de banda
- Reduce el tiempo de respuesta percibido por el usuario

Por su naturaleza una estructura de caché es finita, ya que reside en la memoria principal. Por este motivo, es necesario tener una estrategia o política de reemplazo y / o admisión de los elementos que la componen. Esto es, ítems que en un momento dado tenia sentido conservar, pueden volverse obsoletos y necesitan ser removidos para dar lugar a nuevos ítems. Muchos algoritmos de reemplazo han sido estudiados, que incluían diversos criterios de admisión y/o desalojo [12]. Eventualmente, propuestas de políticas de admisión basadas en algoritmos de aprendizaje automático comenzaron a aparecer en la literatura [15]. Lo que se busca en esos casos es poder predecir, a partir de ciertas características de una consulta, que probabilidad existe de que la misma sea vista en el futuro nuevamente. Esto plantea un escenario que puede ser modelado como un problema de clasificación, donde se clasifiquen consultas que arriban al SE con las etiquetas “Frecuente” o “Infrecuente”, excluyentes entre si.

Incorporar algoritmos de aprendizaje automático como criterios de una política de admisión abre escenarios de exploración con enfoque de ciencia de datos. Esta forma de ver el problema no es novedosa [11] [10], pero aún quedan muchas oportunidades de mejora. En el presente trabajo nos proponemos explorar algunas de estas oportunidades en cuestiones que consideramos , aun no han sido estudiadas en profundidad.

1.1. Fundamentos del problema

El crecimiento de usuarios con exigencias de tiempos cortos y buena calidad de respuestas es una realidad cotidiana de muchos sistemas, incluyendo los motores de búsqueda web. Cualquier posibilidad de optimización de los procesos puede redundar en mejor experiencia de usuario, reducción de costos, ahorro energético, por nombrar algunas ventajas. Uno de los puntos de mejora de los motores de búsqueda es hacer una utilización eficiente de la memoria caché de respuestas [3]. Esta memoria ahorra tiempo de computo guardando temporalmente resultados solicitados ante la suposición de que nuevos usuarios los requerirán en el futuro.

En los últimos años ha surgido la posibilidad de utilizar los procesos propios de la minería de texto y la ciencia de datos para tratar de predecir cuando una consulta tendrá una frecuencia que merezca la pena ser almacenada en la estructura caché [11]. Esta información podría aportar a la decisión de si una consulta merece ser agregada a la estructura en memoria o por el contrario es conveniente no considerarla como candidata.

A partir de este escenario, resulta de interés explorar que tan eficiente puede ser un proceso de minería de datos sobre un registro de consultas reales de las cuales se tiene calculadas un conjunto de características. Se plantea entonces la realización tareas de análisis de datos, pruebas de modelos clasificación, optimización de hiperparámetros y por último, evaluación de resultados de los procesos predictivos.

1.2. Objetivos del Trabajo

La hipótesis de partida es que mientras mejor sea el modelo que prediga consultas frecuentes, el impacto en el uso eficiente de la estructura caché será positivo. Sin embargo, en el trabajo actual nos limitaremos a la evaluación respecto al proceso de clasificación en sí mismo sin entrar en evaluación de la efectividad de la memoria caché.

La pregunta de investigación principal de este trabajo es la siguiente:

¿Qué ventajas ofrecen las técnicas modernas de las ciencias de datos en el rendimiento de la predicción de consultas frecuentes en motores de búsqueda web en comparación con los métodos de clasificación clásicos?

De la pregunta de investigación anterior surge el objetivo principal de este trabajo:

Evaluar y comparar técnicas de clasificación de aprendizaje automático sobre conjunto de datos de registros de consultas para determinar si serán o no frecuentes, utilizando técnicas clásicas modernas de minería y ciencia de datos.

Del anterior objetivo nos planteamos los siguientes objetivos secundarios:

- Realizar un análisis exploratorio de datos sobre el *dataset* para entender el mismo e identificar la necesidad de tratamiento previo a los algoritmos de aprendizaje automático.
- Ejecutar un modelo de clasificación de los algoritmos `RandomForest` y `LightGBM` con los hiperparámetros por defecto.
- Realizar una optimización de hiperparámetros sobre ambos modelos.
- Ejecutar un modelo de clasificación de ambos algoritmos en base a los mejores hiperparámetros hallados por el proceso de optimización.
- Evaluar y comparar resultados.
- Identificar oportunidades de mejora que den lugar a futuros trabajos.

1.3. Estructura del Documento

El presente trabajo se estructura de la siguiente manera. En la sección 2 se establece el marco teórico y los fundamentos del presente trabajo. Se repasan brevemente los trabajos previos que dan sustento al objetivo principal que nos hemos planteado. La sección 3 establece los modelos a utilizar, define y analiza el *dataset* elegido y propone las métricas a ser evaluadas, finalizando con una explicación general del diseño de pruebas realizado. En la sección 4 se analizan los resultados de la ejecución de los modelos, explicando los hallazgos obtenidos. Por último, en la parte 5 esbozamos algunas conclusiones de todo el trabajo a modo de cierre y dejamos asentados algunos posibles trabajos futuros.

2. Trabajos Relacionados

En la presente sección abordaremos los fundamentos teóricos del trabajo y se establecen de forma somera los antecedentes que dan lugar a la pregunta de investigación. Primero abordaremos la problemática de la memoria caché en los motores de búsqueda (subsección 2.1), luego repasamos los modelos de aprendizaje automático elegidos para los experimentos para terminar analizando los antecedentes de uso del enfoque de ciencia de datos en la administración de memorias caché (subsección 2.2)

2.1. Caché de resultados en motores de búsqueda

Como ya se mencionó, entre los diversos aspectos que se pueden mejorar para hacer escalables e incrementar la eficiencia de los procesos en los motores de búsqueda web (SE por la sigla de su definición del inglés *Search Engines*), uno de ellos es optimizar la capacidad de estos SE de aprovechar el procesamiento de una consulta previamente realizada por un usuario para responder a otro cuya necesidad de información coincida en el futuro. Por ejemplo, es probable que durante noviembre y diciembre del año 2022, muchos usuarios hayan realizado la consulta “mundiales

FIFA” a SE como Google o Bing (por nombrar 2 de los motores mas populares). Este tipo de reutilización de computo realizado previamente se conoce como *caching*, y propone sacrificar un poco de espacio en memoria principal a cambio de no procesar nuevamente los resultados de una consulta almacenandolos en una estructura adicional que contiene estos resultados previamente procesados.

Las técnicas de *caching* están presentes en muchos ámbitos de las ciencias de la computación. En el contexto de este trabajo, definimos a la estructura o memoria caché de resultados como una estructura en memoria de trabajo que almacena respuestas a una consulta que será reutilizada en el futuro, en base a algún criterio de decisión. Existe un consenso en la industria y la academia de que este tipo de estructuras intermedias en los SE mejora el tiempo de búsqueda y reduce simultáneamente la carga de trabajo y el ancho de banda utilizado[1].

Así como es conveniente tener algunos objetos en la caché, es razonable suponer que no conviene tener un gran numero de ellos. Existe un compromiso entre el tamaño de una caché y su efectividad. Si la memoria caché posee una gran cantidad de ítems es ineficiente para la búsqueda de uno de ellos en particular y una caché con baja capacidad para almacenar ítems puede omitir retener elementos que sería útil que sean recordados. Además del tamaño de la caché, otro punto importante es como se toma la decisión de que ingresa y que sale de la caché. Existen diversos algoritmos para tomar la decisión de que ingresa y que es eliminado en este tipo de estructuras, que son llamados también políticas de caché.

Diversas políticas de caché han sido estudiadas a lo largo de los años. Se considera que una caché es estática cuando la decisión de que elementos serán parte de la misma es tomada en un proceso previo al uso de la estructura. Decimos que una caché es dinámica cuando la definición de que ítems la integran y cuales deben ser retirados se toma en tiempo de ejecución (es decir, a medida que las consultas van llegando al motor de búsquedas). En general, existen un acuerdo de que estructuras híbridas son el enfoque que muestra mejor rendimiento, donde parte de la memoria caché es estática y parte dinámica[6]. Cualquier política de caché podrá tomar en cuenta diferentes criterios a la hora de definir si un elemento pertenecerá o no a la memoria, es decir, la política de reemplazo. Existen muchos criterios que se basan en diferentes hipótesis de comportamiento de los elementos [14]. Por ejemplo, para el caso de consultas en SE, un criterio podría ser que si un ítem fue visto varias veces en el ultimo tiempo, entonces existe cierta probabilidad de que vuelva a aparecer en el corto plazo. Es una hipótesis donde la popularidad reciente determina la aparición en el futuro. No es el único criterio para definir esto, pero si es uno de los mas conocidos.

En cualquier caso, esta claro que poder determinar si una consulta será frecuente o no en el futuro es un dato de relevancia que podría ayudar a manejar las estructuras de caché de forma eficiente.

2.2. Modelos de aprendizaje automático para políticas de caché

Los algoritmos de aprendizaje automático se han ido incorporando a un gran número de áreas en la última década. Las mejoras constantes en la capacidad de computo acompañadas por reducciones de costos en el *hardware* o acceso económico a *hardware* en la nube ofrece oportunidades de incorporar estrategias de la ciencia de datos a cualquier problema. Esto a ha generado la aparición de algoritmos que proponen enfoques novedosos a problemas clásicos, como por ejemplo problemas de clasificación. En particular, se observa una presencia importante de estrategias de ensamble aprovechando un modelo simple como es el árbol de decisión. Los ensambles proponen que una combinación lineal ponderada de varios modelos construidos a partir del mismo *dataset* generan un meta-clasificador que rinde mejor que los modelos evaluados de forma individual [5].

En particular, dos métodos clásicos para realizar ensambles son algoritmos que se agrupan bajo la estrategia de *bagging* y el grupo que se conocen como *boosting*. En resumidas cuentas, las estrategias de *bagging* (acrónimo de *bootstrap aggregation*) propician una reducción de la varianza del modelo final en función de las estrategias de construcción de los distintos clasificadores. Por ejemplo, para el caso del modelo **RandomForest**, se proponen estrategias de sub-muestreo del conjunto de entrenamiento, de manera que diferentes arboles de decisión son entrenadas con porciones distintas del conjunto de datos[8]. Por su parte, el *boosting* propone una secuencia de modelos donde cada uno de ellos utilizan los residuos de pasos anteriores de la secuencia, dando lugar a una reducción del sesgo de clasificación[7]. Esta estrategia ha incorporado dos algoritmos en el último tiempo, el popular **XGBoost** [4] y una versión optimizada y eficiente conocida como **LightGBM** [9]. Este último algoritmo introduce mejoras al *boosting* al realizar evaluaciones de la cantidad de información que aporta una muestra y decide de forma dinámica si lo tiene en cuenta o no en un paso de la iteración. El efecto de esto es que los sucesivos arboles solo ven un subconjunto del *dataset*, mejorando la eficiencia con un costo bajo al ser una estrategia aproximada y no exacta.

Algoritmos de este tipo ya han sido utilizados para tareas de clasificación de registros de consultas. En [15] se plantea, para una caché de intersecciones (no de resultados), detectar los registros frecuentes e infrecuentes de una base de datos utilizando los modelos **Naive Bayes**, **Arboles de decisión** y **RandomForest**, pero no reporta optimización de hiperparámetros de ningún tipo. En 2017 Kucukyilmaz plantea la clasificación de consultas según su frecuencia de aparición haciendo uso de **Gradient Boost Machine Tree** [11], pero con un enfoque basado en optimizar el *hit-rate* de la caché mas que en obtener un buen modelo predictivo. Por su parte, Tonin Monzon y otros plantean un problema similar al nuestro comparando los algoritmos de arboles de decisión **C4.5** versus **Hoeffding Adaptive Tree** (HAT). En este caso, sin embargo, el enfoque esta orientada a evitar el re-entrenamiento cuando existe un cambio de concepto en las consultas. El algoritmo HAT mantiene información estadística de sus hojas, de manera que cuando existe un cambio de concepto, el árbol se auto modifica sin la necesidad de entrenar el modelo nuevamente (va aprendiendo de las nuevas muestras en la fase de predicción) [16]. En este caso, si bien reportan algún tipo de optimización de hiperparámetros, esta claro al usar arboles de decisión y no ensambles que el foco tampoco esta puesto en obtener modelos predictivos competitivos con el estado del arte. Por su naturaleza, el algoritmo HAT no ve todo el *dataset* sino que define una ventana pequeña, para entrenar además los cambios de concepto y plantea un compromiso entre la potencia predictiva menos eficiente a cambio de una adaptación que evita entrenamiento offline.

Por lo estudiado hasta ahora, no se ha encontrado en la literatura un foco puesto en tratar de determinar cual es la capacidad de modelos de ensamble con buena optimización para determinar la frecuencia a futuro de una consulta basado en el historial pasado del SE. Como se ve, el problema que nos planteamos no es novedoso, sin embargo, acá nos interesa verlo como un problema predictivo y no como un problema de eficiencia de la estructura caché. Nuestro foco va a estar en explorar modelos exhaustivamente y compararlos para determinar si es posible construir un buen clasificador de nuestro registro de consultas.

3. Materiales y Métodos

En esta sección describiremos en detalle en que consiste el conjunto de datos seleccionado para los experimentos realizados, incluyendo un análisis exploratorio de datos (subsección 3.1). Luego se explicaran los criterios de utilización de los modelos predictivos y su configuración, así como también la forma en que se ejecutó la exploración de los hiperparámetros, terminando este análisis con las métricas que serán reportadas (subsección 3.2). Para terminar, se discute brevemente como se estructuran todos los experimentos llevados a cabo para el trabajo (subsección 3.3).

3.1. Conjunto de datos

En el año 2006, una popular compañía del momento que ofrecía como uno de sus servicios un motor de búsquedas pone a disposición de la comunidad un conjunto de datos que contenía el registro de consultas en crudo de los usuarios durante un rango de 3 meses. Cada registro de la base de datos consta de la cadena de texto de la consulta o búsqueda, una marca de tiempo, un indicador de sesión y alguna información adicional. De dicho conjunto de datos original, se plantea una propuesta de *dataset* derivado [11] donde muchas características adicionales son calculadas, algunas a partir de operaciones sobre las columnas originales y otras a raíz de información externa. Para este trabajo se tuvo acceso a una versión de este *dataset* derivado con la mayoría de las columnas disponibles (Ver apéndice A para ver en el cuadro 6 listado completo de las columnas del *dataset* y su definición). En todos los casos, cada registro tiene asignado alguna etiqueta de las 2 posibles para la clase que buscamos: **NoFrecuente** o **Frecuente**. La definición de **NoFrecuente** se aplica para el caso de que la consulta solo aparezca una vez en todo el *dataset*. Si aparece al menos 2 veces, se la cataloga como **Frecuente**. La pertenencia a una de las clases es exclusiva para un registro dado. Además se cuenta con un id del registro, un id de usuario y el texto de la consulta original.

Se explicaran a continuación los tipos de características que se pueden encontrar:

- Características de las consultas: Son *features* calculadas a partir de identificar ciertos patrones en la cadena de texto de la consulta o estadísticas de las consultas en si.
- Características de la sesión: Son derivadas del análisis de las sesiones de usuario en particular.
- Características del índice: Son características relacionadas a una colección de documentos de referencia (*ClueWeb12* [2]).

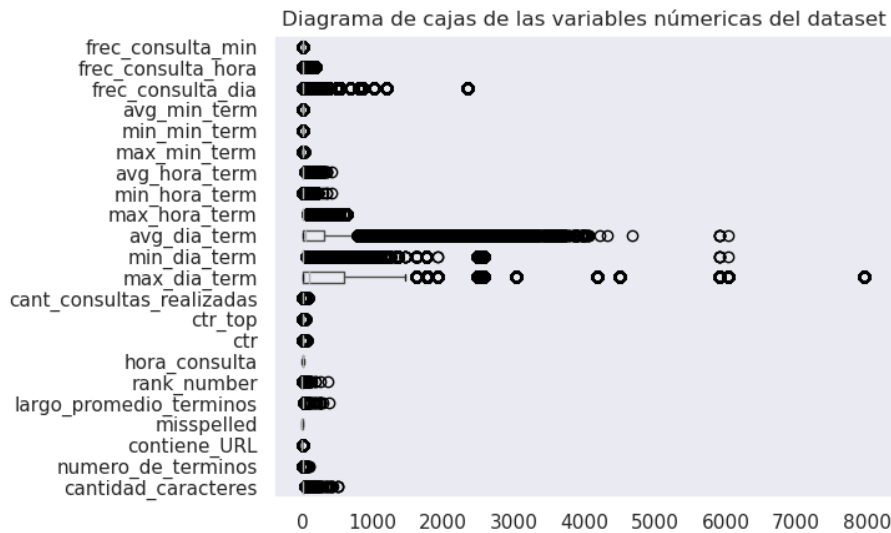


Figura 1: Boxplot de todas las *features* del *dataset* sin ningún tipo de normalización. Se puede observar una fuerte presencia de *outliers* producto de la naturaleza sesgada positivamente de la mayoría de las variables del *dataset*.

- Características de términos: Son *features* calculadas a partir de estadísticas de los términos en el log de consultas y ventanas de tiempo.

El conjunto de datos cuenta con un total aproximado de 20 millones de registros. Sin embargo, se definió de cara a estos experimentos trabajar con varios *sub-datasets* recortados de los mismos. Estos recortes se construyeron siempre desde el primer registro y de forma continua hasta:

- *Dataset 1*: Hasta la consulta número diez mil
- *Dataset 2*: Hasta la consulta número veinte mil
- *Dataset 3*: Hasta la consulta número cuarenta mil
- *Dataset 4*: Hasta la consulta número ochenta mil
- *Dataset 5*: Hasta la consulta número ciento sesenta mil
- *Dataset 6*: Hasta la consulta número doscientas mil

De esta forma, el *dataset 1* esta incluido en todos los demás, el *dataset 2* esta incluido en todos los que le siguen, etcétera. No se hizo *sampling* aleatorio para no afectar el factor temporal y secuencial de los registros.

3.1.1. Análisis exploratorio de datos

Este análisis se realiza sobre el *dataset 6*, que incluye a todo el resto por la forma en que son contruidos, tal como se explicó en la sección anterior. El conjunto de datos fue analizado con el lenguaje de programación Python y utilizando todas las librerías estándar de ciencia de datos: Pandas, Numpy y Matplotlib, entre las mas conocidas.

El *dataset* cuenta con 27 columnas. Un listado completo se puede ver en el cuadro 6. Para el análisis exploratorio es necesario excluir las columnas *id_consulta*, *id_usuario*, *query_string* y *hora_de_la_consulta*, que son las que permiten identificar la consulta dentro del espacio temporal y por usuario. Para el análisis también se dispone de la columna *target* o clase, que servirá para la tarea de clasificación. Para el análisis entonces quedan 22 variables.

Estas 22 variables son de tipo entero. Sus rangos de valores son muy variados, lo que puede observarse en el diagrama de cajas de la figura 1. En la misma, además de observar la presencia de valores atípicos en muchas de las columnas, es visible la observación de rangos muy diferentes.

La *feature cantidad_caracteres* es el tamaño, en letras, de la consulta. Así por ejemplo, si el usuario ingresó *casa perro*, el valor de la columna será 10 (se incluyen los espacios en el conteo).

La columna *numero_de_terminos* cuenta la cantidad de palabras presentes en la consulta. Así por ejemplo, usando la misma consulta de ejemplo *casa perro*, el valor de esta *feature* sería 2. El rango de esta columna, por su naturaleza, es bastante chico. Cubre valores entre 1 y 27, sin incluir la totalidad de valores intermedios.

En el caso de las variables `mispelled` y `contiene_URL`, si bien son numéricas, en realidad son columnas booleanas ya que su rango son 2 valores, cero y uno. La primera establece si la consulta contiene algún error ortográfico, mientras que la segunda establece si la en la consulta existe alguna dirección web entre sus términos.

Para la columna `largo_promedio_terminos` se mira cada palabra de la consulta, se calcula el largo de todas y se hace una media aritmética. Así, por ejemplo, para nuestra consulta de ejemplo utilizada anteriormente, el valor de esta columna sería 4,5. Su rango va entre 1 y 255, pero no todos los valores intermedios están presentes.

Una serie de características se construyeron alrededor de la consulta y su relación con el motor de búsqueda. En el caso de la *feature* `rank_number`, se trata de la posición del primer resultado elegido por el usuario. Es obvio que entre los valores validos estará el cero, lo cual significa que el usuario no eligió ningún resultado. Para el caso de la columna `hora_consulta`, simplemente se extrae la hora de la columna de marca de tiempo del conjunto de datos original.

Otro subgrupo de variables son calculadas en relación a la sesión. Se entiende como sesión en el contexto de este *dataset* a un mismo usuario interactuando en un rango de tiempo determinado con el motor de búsquedas. El *dataset* posee una característica que nos indica cuando un grupo de consultas fueron realizadas por el mismo usuario. Las columnas `ctr`, `ctr_top` y `cant_consultas_realizadas` son calculadas en base a este concepto. La primera refleja la cantidad de resultados elegidos en la sesión. El segundo, la cantidad de resultados elegidos en la primer posición. Por último, tenemos un recuento del número de consultas que realizó ese usuario.

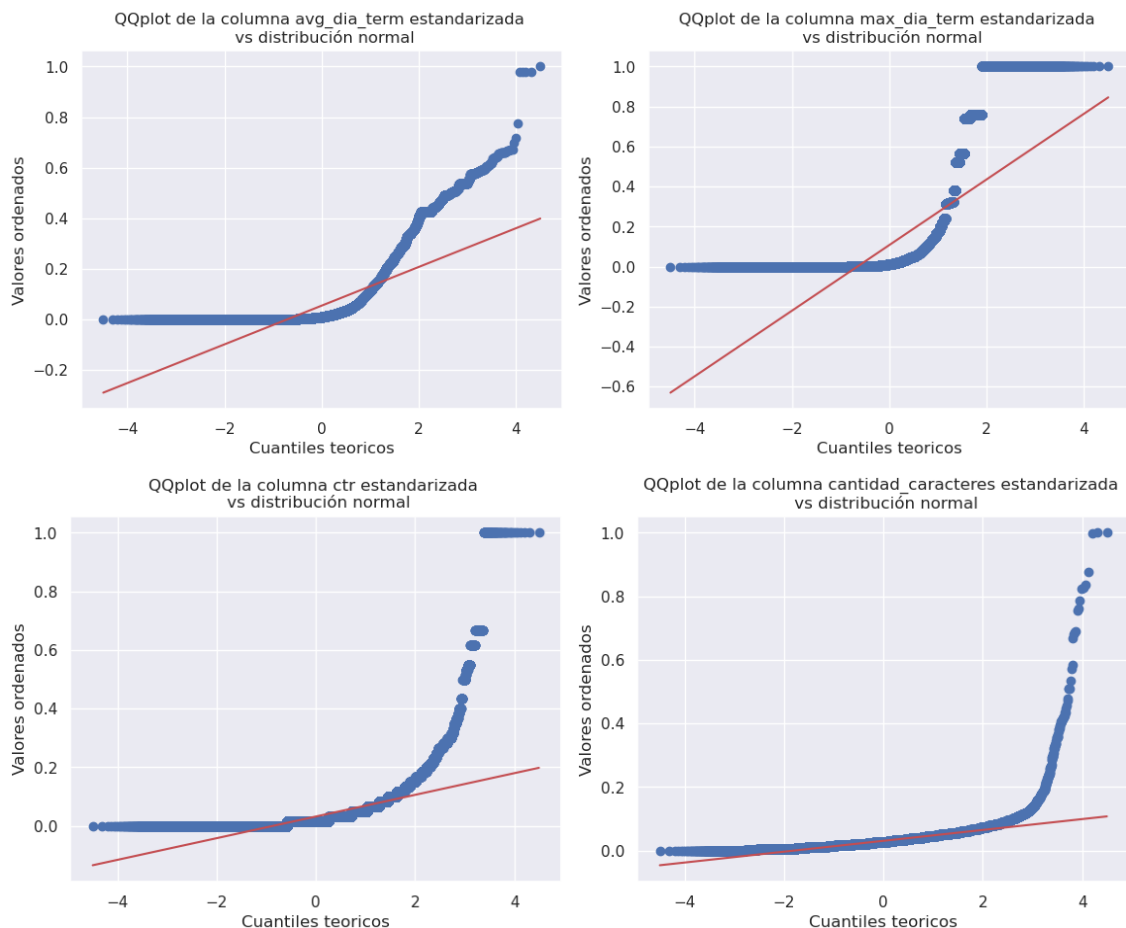


Figura 2: QQplots de columnas estandarizadas. En todos los casos se observan variables sin normalidad.

Muchas de las variables del *dataset* se construyeron a partir de un mecanismo de ventanas móviles de un registro respecto a su pasado. Entre ellas, tenemos las columnas `frec_consulta_dia`, `frec_consulta_hora` y `frec_consulta_min`. Estas *features* computan la frecuencia de la consulta del registro actual para tamaños de ventana del último minuto, en la última hora y en el último día, respectivamente. Esta ventanas son utilizadas también a nivel de frecuencia de los términos. Las 9 variables que computan esto para las 3 ventanas y para la media, máximo y mínimo de los términos de la consulta se construyeron con esta lógica. Así, por ejemplo, la variable `max_dia_term` toma para el registro actual, cual es la palabra que mas se repitió en el último día (últimas 24

horas).

Según la definición clásica de dato atípico u *outlier*¹, muchas de las variables presentan registros en estas condiciones. Mas aún, en los diagramas de caja que se ven en la figura 1, las variables construidas con ventanas móviles de un día son las que muestran la mayor asimetría (en particular, asimetría positiva). Esto es esperable, ya que los estudios en frecuencias en el área de minería de texto suelen mostrar que los datos adquieren una distribución compatible con la Ley de Zipf, que es una versión discreta equivalente a la distribución de Pareto. Si bien una forma de estandarizar datos con ligera asimetría es mediante un escalamiento, si la distribución es muy asimétrica la transformación no resulta útil. En este caso así fue, dado que se re-escalaron los datos con 2 tipos de técnicas, `MinMaxScaler` y `StandardScaler`. En ambos casos el resultado fue muy similar a los datos de origen en cuanto a la distribución resultante. Por esto mismo se decidió en la figura 1 solo incluir el diagrama de cajas de los datos tal como están en el *dataset*.

Para terminar de confirmar este diagnostico, se realizaron pruebas de normalidad visuales y analíticas de algunas columnas, encontrando, como resulta esperable, la no normalidad de las mismas. En cuanto a las técnicas visuales, se realizaron *QQplots* que comparan la distribución estandarizada de 4 columnas contra una distribución normal. Mientras menos acompañen los datos a la recta roja, mas lejos están los datos de una normal estándar. Se puede observar en la figura 2 este comportamiento en 4 variables elegidas de entre todas las del *dataset* (que comparten el comportamiento de las presentadas en este informe). En cuanto al test analítico, se hicieron tests de Shapiro-Wilks y tests de D'Agostino-Pearson, obteniendo el mismo resultado de no normalidad, sobre todas las *features* de la base de datos. Cabe destacar aquí que en todos los casos da significatividad estadística pero solo se debe a la cantidad de muestras alta (doscientos mil registros). Es de esperar que con el *dataset* completo, estos resultados se mantengan.

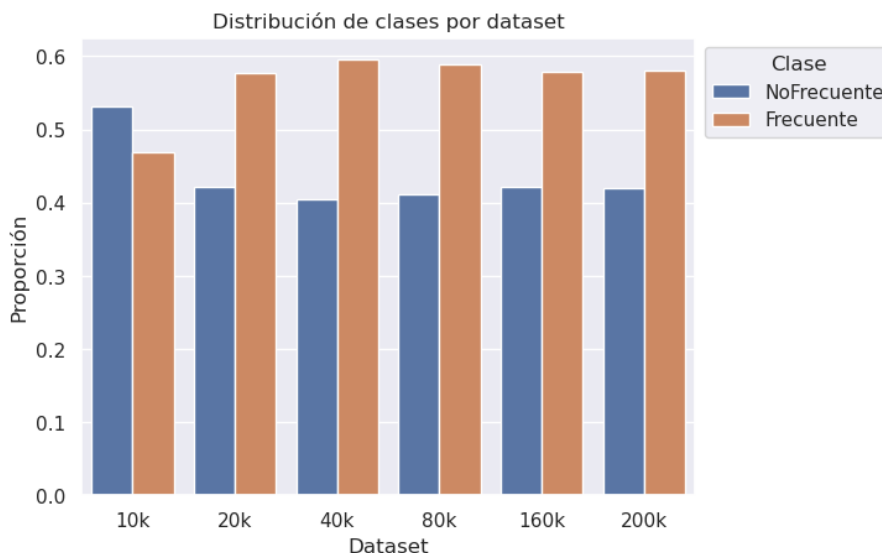


Figura 3: Desbalance de clases por conjunto de datos. Los valores de las columnas se observan en proporción de presencia de la clase. Para cada *dataset*, la suma de ambas columnas suma 1.

Otro estudio que se hizo sobre el *dataset* es la búsqueda de datos nulos o faltantes. Por la forma de construcción de la base de datos, no existen datos nulos y por lo tanto no hay que pensar ninguna estrategia de imputación o exclusión por este motivo.

Se analizó el balance entre las clases. En este caso, se consideró relevante analizarlo para todos los *datasets* y no solo para el de 200 mil muestras. En la figura 3 se puede observar para cada uno de los 6 *datasets*, la proporción de clases **Frecuente** y **NoFrecuente**. Para las bases de datos de 20 mil a 200 mil observaciones, podemos ver proporciones similares, con un desbalance de entre un 0.15 a 0.2 a favor de la clase **Frecuente**. Esta relación se invierte para la base de datos de diez mil ítems. Allí se observa que el desbalance es menor (en el orden del 0.07) y a favor de la clase **NoFrecuente**. Queda fuera de este análisis determinar las causas de este comportamiento, pero esto si será tenido en cuenta al definir las métricas y durante la presentación de los resultados de los experimentos.

En cuanto a la correlación de las columnas, se observa que algunas variables presentan un alto grado de correlación usando la formula de correlación de Pearson. Se define como correlación alta

¹Nos referimos aquí al conocido como test de Tukey, donde se define como valor atípico a todo dato que caiga por afuera (positivo o negativo) del rango intercuartílico multiplicado por 1,5

un número igual o superior a 0.8 o inferior a -0.8. El estudio arrojó que si se decide prescindir de dichas variables, el total de columnas a procesar pasa de 22 a 15 columnas. Se presenta en la figura 4 el correlograma final de estas 15 columnas. Se consideró que en esta fase del problema no era conveniente descartar las variables y se mantuvieron en la base de datos final.

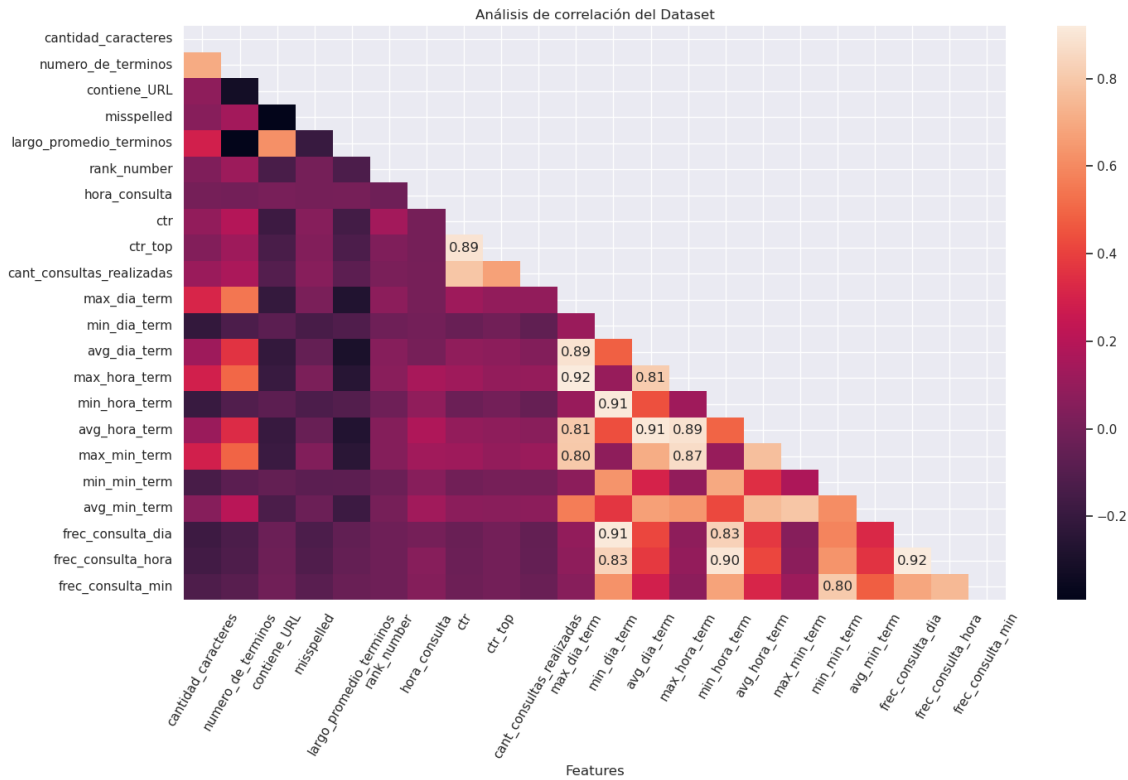


Figura 4: Correlograma de las variables del conjunto de datos. Se calculó la correlación de Pearson para todos los pares de variable. Se muestran los valores de correlación solo para las variables que tienen una correlación considerada alta, es decir, por encima de 0,8 o por debajo de -0,8

3.2. Modelos de aprendizaje automáticos

Se describen a continuación la forma en que los modelos son implementados en los experimentos realizados. Primero se describen los detalles del modelo **RandomForest** (En el apartado 3.2.1), luego se explican aspectos relevantes de la utilización del algoritmo **LightGBM** (apartado 3.2.2). Se continua con una descripción de la estrategia de exploración de hiperparámetros implementada (apartado 3.2.3), para finalizar con las definiciones respecto a métricas a evaluar en los experimentos (apartado 3.2.4). En todos los casos y para reproducibilidad, se configuró en los algoritmos que tuvieran aleatoriedad involucrada la semilla con el valor 42.

3.2.1. RandomForest

Para ejecutar el modelo de árboles de búsqueda aleatorizados o mejor conocido como **RandomForest**, se utilizó la implementación para Python realizada por la librería **Scikit Learn** [13]. Para este modelo, se definió utilizar 2 variantes:

- Versión Base: Versión que utiliza los hiperparámetros por defecto de la librería para construir el árbol.
- Versión Optimizada: Versión que posee los hiperparámetros encontrados por la optimización realizada (ver apartado 3.2.3).

3.2.2. LightGBM

El algoritmo de **LightGBM** utilizado es el desarrollado por *Microsoft* y disponible en el siguiente sitio ². Para esta librería se procedió de forma análoga al algoritmo de **RandomForest** explicado en

²<https://lightgbm.readthedocs.io/en/latest/index.html>

el apartado 3.2.1. De esta forma, primero se generó una versión base con los valores por defecto que aporta la librería y una versión optimizada por una exploración de hiperparámetros.

3.2.3. Optimización de hiperparámetros

Existen varias alternativas de exploración de hiperparámetros. Para estos experimentos se decidió realizar una exploración aleatoria sobre un conjunto de hiperparámetros elegidos de entre los disponibles de las librerías. Para estos experimentos se eligió entonces utilizar el algoritmo `RandomSearchCV` de la librería `Scikit Learn` [13].

Para configurar la exploración, se configuró la semilla en 42, para reproducibilidad, y se dejó por defecto la validación cruzada en *5-fold cv*. La cantidad de iteraciones en la exploración fueron de 100 para pruebas de desarrollo y 1500 en la corrida final.

En el cuadro 1 se puede observar el espacio de hiperparámetros explorado para el modelo `RandomForest`. Como ejemplo de salida de este proceso, se puede observar en el cuadro 2 los valores de los hiperparámetros y el *score* asociado de los primeros 5 mejores resultados. Para la versión optimizada se eligieron los hiperparámetros de la primer fila.

Hiperparámetro	Valores disponibles
n_estimators	100, 200, 300, 400, 500, 600, 700, 800, 900, 1000, 1100, 1200, 1300, 1400, 1500, 1600, 1700, 1800
criterion	'gini', 'entropy'
max_depth	None, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15
min_samples_split	2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26
min_samples_leaf	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14
bootstrap	True, False

Cuadro 1: Espacio de búsqueda de hiperparámetros del algoritmo `RandomForest` para el objeto `RandomSearchCV`. De todas las combinaciones posibles, se exploran 1500 elegidas al azar con la semilla 42.

Una observación que se puede realizar de lo que se observa en el cuadro 2, es que la exploración encontró conjuntos de hiperparámetros que superan la media del modelo base. Para el caso del hiperparámetro `max_depth`, resulta evidente que se necesita seguir explorando, ya que en los 5 casos listados, que fueron los mejores *scores*, es evidente que siempre están en el tope máximo de valores permitidos.

rank_test_score	1	2	3	4	5	Base	Low	Right	Observaciones
n_estimators	200	400	400	600	500	100	100	1800	Salto de 100 en 100
min_samples_split	4	16	4	14	26	2	2	26	A partir de números pares
min_samples_leaf	2	10	2	9	6	1	1	14	
max_depth	15	15	15	14	15	None	2	15	Se incluye None
criterion	entropy	entropy	entropy	entropy	gini	gini	NA	NA	
bootstrap	TRUE	TRUE	TRUE	FALSE	TRUE	True	NA	NA	
mean_test_score	0.83530	0.83514	0.83514	0.83513	0.83511	0.8302	NA	NA	

Cuadro 2: Mejores 5 conjuntos de hiperparámetros encontrados por la exploración del espacio de búsqueda con el algoritmo `RandomSearchCV` para el modelo `RandomForest`. De las 1500 elegidas al azar con la semilla 42, estas son las mejores 5 en base al *score*. Se agrega los valores de hiperparámetros del algoritmo base y de los rangos de generación de valores para facilitar su comparación.

Para el modelo `LightGBM` se siguió una estrategia similar, solo que el objeto de la librería dispone de mucha mayor cantidad de hiperparámetros. La estrategia de exploración del espacio se puede encontrar en el cuadro 3. El objeto `param_generator` es de la clase `numpy.random.default_rng`. Es un generador de números con distribución uniforme de tipo flotante (para el caso de `uniform`) y de tipo entero (en el caso de `integer`). En todos los casos, y sabiendo que esta exploración resulta insuficiente, se generan 10 ejemplos de cada hiperparámetro.

La estrategia de exploración de hiperparámetros elegida pudo hallar un conjunto de valores que superaron el algoritmo base. Esto se puede observar en el cuadro 4. En el mismo se reportan los mejores 5 conjuntos de hiperparámetros y se los compara con el algoritmo base. Con fondo gris se resaltan el conjunto de hiperparámetros utilizados en la versión optimizada y en la versión base, respectivamente. Entre ambas versiones se puede apreciar una diferencia de *score* de 0,00635.

El cuadro 4, además, agrega los rangos bajo y alto de generación aleatoria, para observar si existe tendencia hacia alguno de los extremos en los valores que dieron *score* alto. Se puede hacer una observación especial para el caso del hiperparámetro `min_sum_hessian_in_leaf`, el cual fue generado claramente en rangos no adecuados. Queda como trabajo futuro explorar rangos validos de dicho hiperparámetro.

Hiperparámetro	Valores disponibles
bagging_fraction	param_generator.uniform(low=0.1,high=1.0,size=10)
bagging_freq	param_generator.integers(low=0,high=10,size=10)
feature_fraction	param_generator.uniform(low=0.1,high=1.0,size=10)
max_depth	param_generator.integers(low=3,high=100,size=10)
min_data_in_leaf	param_generator.integers(low=10,high=1000,size=10)
num_leaves	param_generator.integers(low=10,high=100,size=10)
learning_rate	param_generator.uniform(low=0.001,high=0.1,size=10)
lambda_l1	param_generator.uniform(low=0.0001,high=1,size=10)
lambda_l2	param_generator.uniform(low=0.0001,high=1,size=10)
min_sum_hessian_in_leaf	param_generator.integers(low=20,high=60,size=10)
n_estimators	param_generator.integers(low=100,high=1000,size=10)
max_bin	param_generator.integers(low=20,high=400,size=10)

Cuadro 3: Espacio de búsqueda de hiperparámetros del algoritmo **LightGBM** para el objeto **RandomSearchCV**. De todas las combinaciones posibles, se exploran 1500 elegidas al azar con la semilla 42.

Para finalizar con este análisis, se considera necesario hacer una advertencia. En el modelo **LightGBM** existen muchos hiperparámetros adicionales a los explorados en este trabajo y los rangos validos también varían mucho. Además, generar 10 muestras de cada caso es una restricción extra a los fines de acelerar los experimentos. Debe quedar claro que aún realizando una búsqueda *grid* (completa) sobre los valores generados, se puede estar muy lejos de valores óptimos haciendo uso de esta estrategia.

rank_test_score	1	2	3	4	5	Base	Low	Right
num_leaves	69	98	53	53	69	31	10	100
n_estimators	211	792	168	340	340	100	100	1000
min_sum_hessian_in_leaf	20	54	24	28	20	1e-3	20	60
min_data_in_leaf	20	461	461	20	269	20	10	1000
max_depth	98	3	50	12	50	-1	3	100
max_bin	385	294	234	234	385	255	20	400
learning_rate	0.058	0.090	0.058	0.040	0.040	0.1	0.001	0.1
lambda_l2	0.184	0.208	0.239	0.968	0.184	0	0.0001	1
lambda_l1	0.109	0.109	0.252	0.350	0.215	0	0.0001	1
feature_fraction	0.826	0.675	0.826	0.545	0.448	1.0	0.1	1.0
bagging_freq	8	5	9	9	1	0	0	10
bagging_fraction	0.939	0.824	0.797	0.665	0.824	1.0	0.1	1.0
mean_test_score	0.83708	0.83700	0.83698	0.83689	0.83686	0.83073	NA	NA

Cuadro 4: Mejores 5 conjuntos de hiperparámetros encontrados por la exploración del espacio de búsqueda con el algoritmo **RandomSearchCV** para el modelo **LightGBM**. De las 1500 elegidas al azar con la semilla 42, estas son las mejores 5 en base al score. Se agrega los valores de hiperparámetros del algoritmo base y de los rangos de generación de valores para facilitar su comparación.

3.2.4. Métricas a ser evaluadas

Como es habitual en experimentos de esta naturaleza, se reportarán los siguientes resultados:

- Matriz de confusión para explorar la capacidad predictiva de los modelos
- Métrica F1 debido al desbalance entre las clases para todos los *datasets* vistas en el análisis exploratorio.
- Área bajo la curva ROC, para evaluar y comparar el rendimiento de los modelos.

3.3. Organización de los experimentos

En primer lugar, se armaron las 6 versiones del conjunto de datos descrita en 3.1. Luego se realizó el análisis exploratorio de datos explicado en el apartado 3.1.1 sobre el *dataset* de 200 mil muestras. A partir de eso se definió que era conveniente conservar todas las columnas, que no existían datos nulos para imputar y que existe un desbalance en todos los *datasets* a tener en cuenta en el análisis. A continuación se entrenan las versiones base de ambos modelos y se los prueba. Para esto se hace una división de la base en proporción 80/20 (entrenamiento y pruebas

respectivamente). Para finalizar, con los resultados, se generan cuadros y figuras para los resultados que se analizarán en la sección 4.

En total, se ejecutaron 24 experimentos. Cada uno de estos experimentos representa una combinación diferente de los siguientes aspectos:

- Para cada uno de los 6 *datasets* 10k, 20k, 40k, 80k, 160k y 200k.
- Para cada uno de los 2 modelos *RandomForest* y *LightGBM*.
- Para la versión base y versión optimizada de cada modelo.

4. Resultados

A continuación se presentan los resultados de los experimentos detallados anteriormente. Primero se analizan las matrices de confusión de experimentos seleccionados y luego se reporta la métrica F1 para dichos modelos. En el anexo B se reportan otras métricas adicionales. Luego se muestran las áreas bajo las curvas ROC para los modelos analizados previamente y se realiza una comparativa entre dichas áreas para los diferentes experimentos. En el anexo C se reportan todas las curvas ROC que dan origen a las áreas estudiadas.

4.1. Matrices de confusión

En la figura 5 se pueden observar las matrices de confusión del modelo **RandomForest** resultantes de correr las versiones base y optimizada sobre el conjunto de datos 200k. De acuerdo a la división 80 / 20, estas matrices contienen 40 mil ejemplos que son los que se encuentran incluidos en el conjunto de pruebas. Lo primero que se puede observar es que la versión base tiene mejor poder de predicción para la clase **Frecuente** que la versión optimizada. Esta última, a su vez, gana prediciendo la clase **NoFrecuente**. Esto es evidente comparando las diagonales de la figura 5. Como complemento, resulta claro que el modelo optimizado empeora al predecir como **NoFrecuente** algunos elementos de la clase **Frecuente** y gana en el inverso (comparando las esquinas inferiores izquierdas de las matrices).

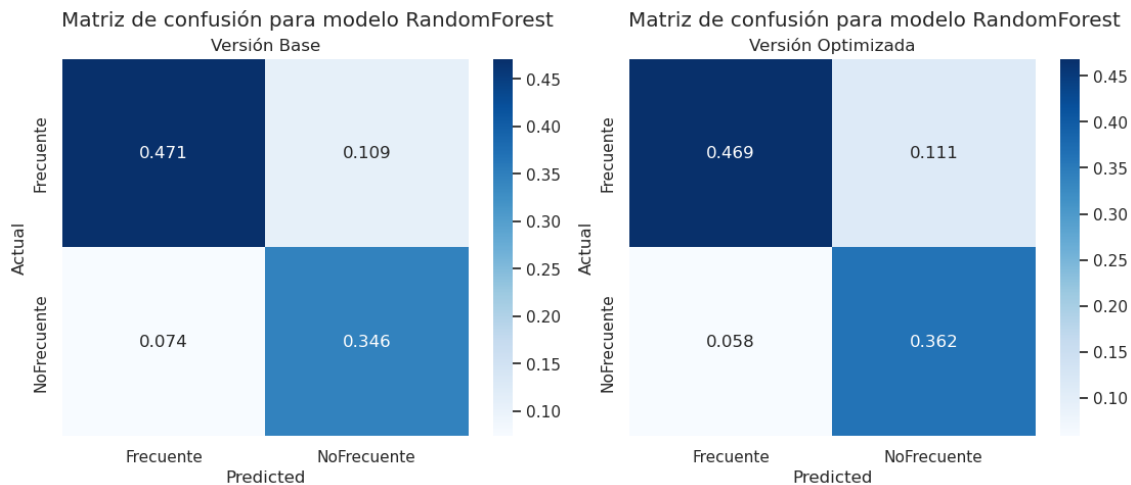


Figura 5: Matrices de confusión normalizadas para el modelo **RandomForest** para el *dataset* 200k para la versión base y versión optimizada.

En este punto, es relevante realizar un análisis de si es indistinto o no el aumento de falsos positivos y / o falsos negativos. En el contexto de nuestro problema, predecir como **NoFrecuente** una consulta que si volverá a aparecer, tiene como consecuencia volver a procesar una búsqueda realizada anteriormente, lo cual indica el escenario de falsos positivos. El caso inverso, donde el modelo diga que una consulta es **Frecuente** pero no lo era, solo se pierde un espacio en la memoria caché. Este es el escenario de falso negativos. Resulta claro entonces que el primer caso tiene peores consecuencias que el segundo y por lo tanto, es menos deseable. Siguiendo con el razonamiento, se puede observar que, para los falsos positivos, comparando el modelo base y el modelo optimizado el error se incrementa en 0,002. Por el contrario, en el caso de los falsos negativos, el error disminuye 0,016 puntos. Sería deseable un modelo optimizado que invierta esta secuencia, dado que nos

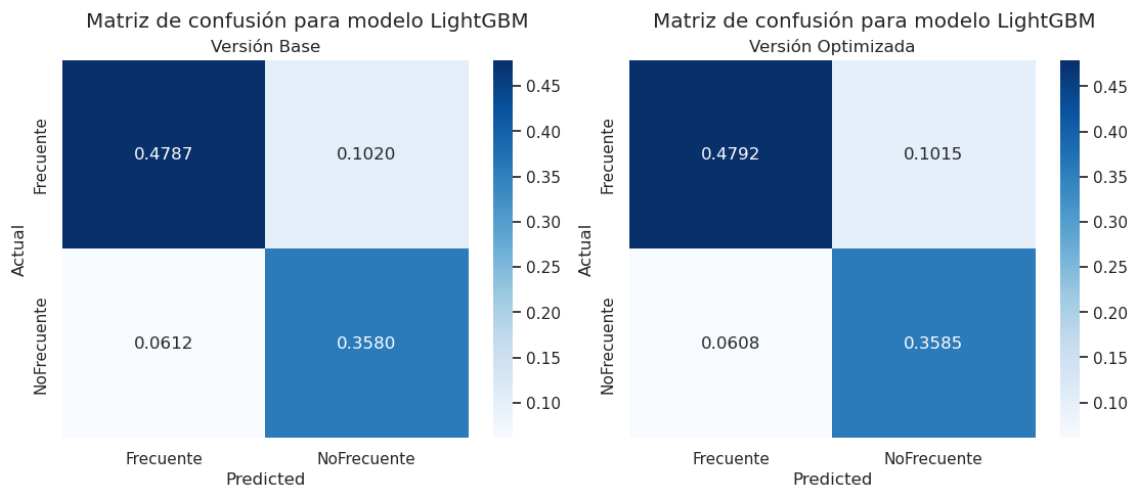


Figura 6: Matrices de confusión normalizadas para el modelo **LightGBM** para el *dataset* 200k para la versión base y versión optimizada.

podemos permitir algunos falsos negativos pero cada falso positivo tiene un costo enorme para nuestro sistema.

Una situación diferente se da para el modelo **LightGBM** y sus matrices de confusión resultantes. En la figura 6 se pueden observar las matrices de las versiones base y optimizada de este modelo. En este caso, el modelo optimizado logró mejorar en toda la diagonal las proporciones de predicción. Es necesario advertir que esta mejora fue, al menos, escueta o de muy bajo impacto. Prueba de ello fue la necesidad de marcar las proporciones con 4 decimales en lugar de 3 para que se vean las diferencias por redondeo. En términos cuantitativos, la cantidad de ítems que el clasificador avanzado logra clasificar bien respecto al base son 20 para el caso de la clase **Frecuente** y 19 ejemplos para el caso de **NoFrecuente**. Estos números no pueden ser extraídos de la figura 6 ya que allí están en números relativos y no absolutos.

Entonces, se puede ver que para este caso si se pudo conseguir un clasificador optimizado que mejora en ambos tipos de errores al clasificador base. Si a esto se le suman las reservas realizadas oportunamente respecto al poco espacio de búsqueda de hiperparámetros explorado para el modelo **LightGBM**, como se comentó hacia el final del apartado 3.2.3, está claro que este problema merece ser abordado de forma mas exhaustiva por esta vía.

4.2. Métricas de rendimiento

Como se planteó en el apartado correspondiente (ver 3.2.4), la métrica relevantes para ser discutida en esta sección será el *score* F1. Esto se debe principalmente al desbalance existente en la distribución de las clases de todos los *datasets*, visto en la figura 3. En estos escenarios de desbalance de clases, reportar las métricas *precision* y *recall* por separado suelen tener inconvenientes de interpretación, ya que ponderan cierto aspecto del clasificador por sobre otros. La métrica *score* F1, por construcción, permite una comparación que pondera los aspectos de las otras dos métricas mencionadas y nos ofrece un único indicador para comparar clasificadores.

En la figura 7 se observan, para todos los experimentos realizados, los diferentes valores de *score* F1. En el caso de **RandomForest**, es claro que en la comparación de ambas versiones para un mismo *dataset*, la opción optimizada supera a la estrategia base. Esto no se verifica en el algoritmo **LightGBM**, donde en las pruebas sobre los conjuntos de datos 40k y 160k se observa que el modelo base supera ligeramente a los modelos optimizados. Esto generó la necesidad de revisar los experimentos realizados, sobre todo el proceso de optimización de hiperparámetros. Se comprobó que hubo un error de configuración en el algoritmo, dejando por defecto el cálculo del *scoring*. Esto generó que la métrica a optimizar fuera el *accuracy* en lugar del F1. Esto explica lo observado en la figura 7, gráfico inferior.

Otra observación que se puede realizar, para ambos modelos y en todas las versiones, es la tendencia al incremento del *score* F1 en la medida que el *dataset* crece. Esto debe ser relativizado dado que el rango de crecimiento del parámetro es muy pequeño. Por ejemplo, para el caso de **LightGBM**, el menor valor de toda la serie es 0,8142 (para el *dataset* 10k da el menor valor en el modelo optimizado) y el mayor es 0,856 (para el *dataset* 40k y la versión base del algoritmo), dando un rango de solo 0,0418. Un análisis similar puede realizarse sobre el modelo **RandomForest**, donde

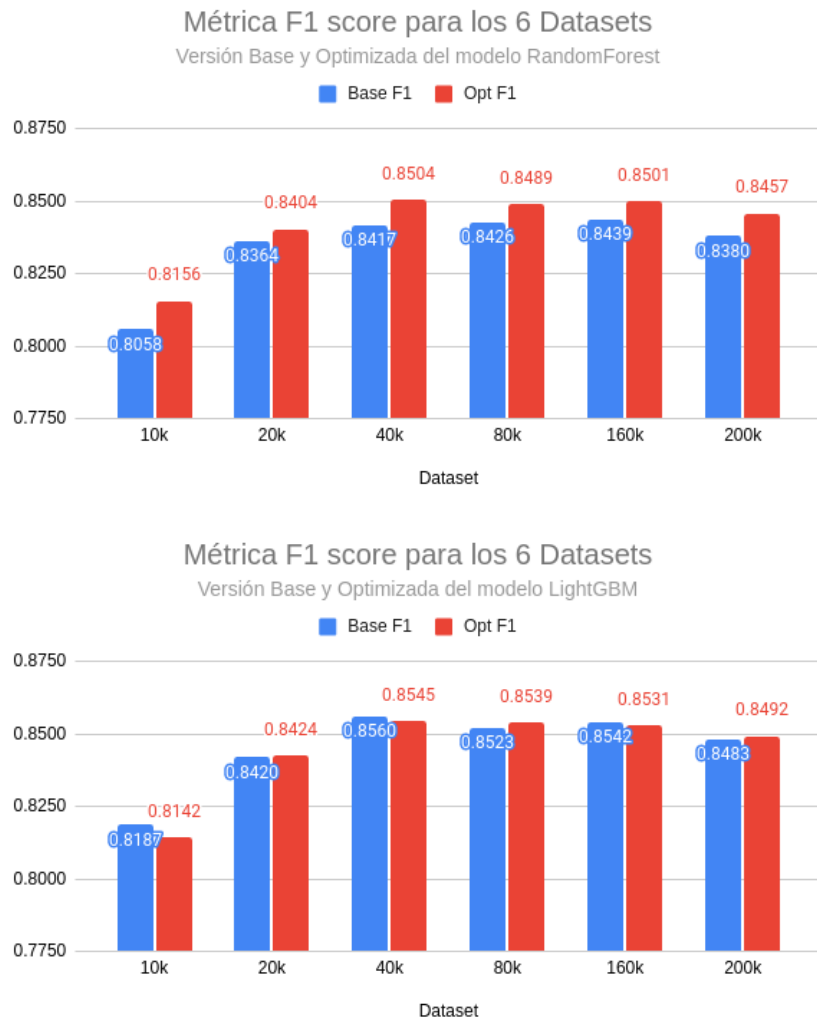


Figura 7: Métrica *score* F1 para diferentes *datasets* en ambos modelos para cada una de las versiones. Cada barra representa un experimento. El eje y se encuentra recortado para mejorar la comparación y se mantienen las escalas en ambos gráficos para hacerlos comparables visualmente.

el rango es de 0,0446 que es similar al analizado para **LightGBM**.

Por último, se hace observar que comparando entre ambos gráficos de la figura 7, el que respecta a algoritmo **LightGBM** (en la parte inferior) supera siempre a sus experimentos equivalentes con **RandomForest**, salvo para el caso del *dataset* 10k y la versión optimizada. Este resultado se relativiza dado el error comentado un par de párrafos atrás respecto al bug en la optimización de hiperparámetros. Además es el único caso donde se invierte el hecho de que **LightGBM** supere a **RandomForest** en lo que respecta a la comparación en base al *score* F1.

4.3. Curvas ROC y área bajo la curva

Las curvas ROC de los modelos resultaron ser bastantes similares. Es esperable dado que los modelos rindieron de forma similar. Sin embargo, nos interesa profundizar en las pequeñas diferencias entre cada uno de ellos a fin de poder encontrar si alguno de los modelos es superior al resto, aunque sea en alguna de sus versiones. Podemos visualizar una comparación de todas las áreas bajo las curvas ROC en la figura 10. Consideramos que resulta mas provechoso visualizar las diferencias entre curvas ROC de esta manera y no superponiendo todas las curvas ROC en una figura que no llegaría a mostrar las pequeñas diferencias entre modelos.

Varias observaciones se pueden realizar de estos resultados. El primer resultado evidente es que todos los modelos tuvieron rendimientos muy similares. Nos referiremos a números que construyen la figura 10 y que pueden ser observados mirando el cuadro 5. La menor AUC reportada fue de 0,8988, para el conjunto de datos 20k y el modelo base de **RandomForest**, mientras que el mayor valor fue 0,9242, dando un rango de variación de todas las AUC de solo 0,0254.

Otra observación que se puede hacer a partir de la figura 10 es que en la medida que el *dataset*

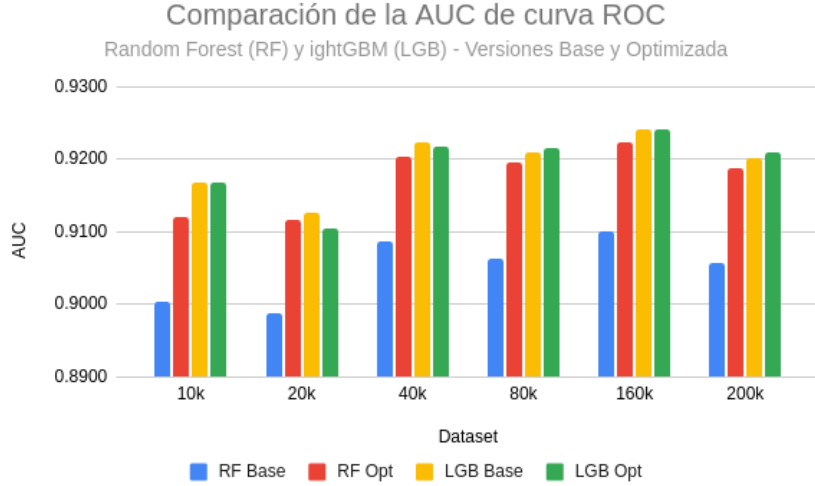


Figura 8: Comparación de las AUC de todos los experimentos. El eje y está ajustado a los extremos de los valores para favorecer la visualización de la diferencia. Cada modelo está comparado con entre sus versiones base y optimizada, y a su vez con el otro modelo al lado, agrupados por *dataset*.

crece en tamaño, se observa una ligera tendencia al crecimiento en el valor de AUC. La tendencia parece mantenerse en todos los modelos y versiones, aún en aquellos casos que la tendencia es a la baja, como se observa en los casos de los conjuntos de datos 80k y 200k. Lo siguiente que surge mirando la misma figura es que existe una diferencia clara entre la versión base del modelo **RandomForest** y el resto de los modelos. No tan evidente es que alguna versión del algoritmo **LightGBM** siempre es superior a cualquiera de los modelos de **RandomForest**. Esto se puede observar con las celdas resaltadas del cuadro 5, que reflejan el valor de AUC máximo de cada fila. Existe un solo escenario donde una versión optimizada de **RandomForest** supera a alguna de las versiones de **LightGBM**. Esto se da para RF optimizado en contraste con LGB optimizado en el *dataset* 20k. Otra observación en relación a esta misma base de datos es que son los valores mínimos de AUC de entre todos los experimentos comparando mismos modelos en los otros conjuntos de datos.

Dataset	AUC RF Base	AUC RF Opt	AUC LGB Base	AUC LGB Opt
10k	0.9004	0.9120	0.9169	0.9169
20k	0.8988	0.9117	0.9125	0.9105
40k	0.9086	0.9203	0.9224	0.9218
80k	0.9063	0.9195	0.9210	0.9216
160k	0.9101	0.9224	0.9242	0.9241
200k	0.9057	0.9188	0.9201	0.9209

Cuadro 5: Valores del área bajo la curva ROC (AUC) para todos los experimentos. Se puede observar en negrita el valor más alto para cada *dataset*.

5. Conclusiones

En el presente trabajo, nos preguntamos inicialmente que ventajas podemos obtener de las técnicas modernas de las ciencias de datos para el problema de clasificación de consultas con baja frecuencia en el contexto de motores de búsqueda web para potenciar políticas de admisión en memorias caché. Discutimos sobre que si bien el problema no era novedoso, siempre se enfocó como un problema secundario dentro de la investigación sobre eficiencia de memoria pero no se puso el foco en generar evidencia alrededor del proceso de clasificación en si mismo.

Se pudo encontrar que el *dataset* presenta naturalmente un desbalance en sus clases, el cual se modifica en función de la cantidad de muestras que se toman, pero que a partir de un cierto número de muestras ese valor parece ser estable. Además se encontraron variables del *dataset* que tienen un grado de correlación alto entre si, pero en ningún caso ameritan ser excluidas del análisis (en ningún caso hubo correlación cercana a 1,0 o -1,0). Además, se observaron que las variables nos presentan una distribución normal, sino que tienen un sesgo positivo, lo cual es esperable ya que la distribución teórica de las variables debería ajustarse a una ley de potencia.

Se eligieron dos modelos basados en arboles de decisión pero que trabajan mediante ensambles. Uno de ellos, `RandomForest`, con una estrategia basada en el *bagging* y el otro, `LightGBM`, con un esquema de *boosting*. Se probaron versiones básicas (hiperparámetros por defecto) de ambos algoritmos y se corrieron pruebas sobre 6 recortes del *dataset*. Luego de correr una optimización de hiperparámetros, se volvieron a correr las mejores versiones optimizadas de los algoritmos sobre los mismos conjuntos de datos.

Los resultados mostraron que en el caso de `RandomForest`, el proceso de optimización de hiperparámetros ayudo a encontrar mejores modelos. Para la librería `LightGBM`, el proceso sufrió de algunos percances que limitaron las mejores. Sin embargo, el espacio de búsqueda de hiperparámetros en este último caso fue mucho mas limitado por cuestiones de tiempos de las pruebas. En todos los casos, se comprobó que *datasets* de mayor tamaño generaban modelos finales que rendían mejor, sin importar si eran modelos base u optimizados. Además, estas mejoras se hacían cada vez mas pequeñas en la medida que la base crecía de tamaño.

5.1. Trabajos Futuros

En muchos momentos del presente trabajo se decidió dejar de lado ciertas pruebas que podrían ser de interés para avanzar en el objetivo principal pero que se desviaban de las pruebas planteadas. En la presente sub-sección hacemos un listado no exhaustivo de estos experimentos, que pueden servir como trabajos futuros.

- Replicar experimentos de artículos que encaran nuestro problema con otros modelos predictivos [11] [15] [16] para usar de *baselines* sobre los experimentos hechos sobre este trabajo.
- Reproducir este trabajo con el *dataset* completo o al menos recortes de mayor tamaño, ya que el crecimiento del *dataset* mostró ser beneficioso para los algoritmos de clasificación entrenados.
- Estudiar en profundidad las variables del *dataset*:
 - Explorar la incidencia en los resultados finales de eliminar variables de alta correlación del *dataset*.
 - Derivado de lo anterior, explorar la incidencia de armar variables nuevas a partir de un análisis de componentes principales con las variables de alta correlación y dejar las primeras 3 o 4 componentes como parte del *dataset* en lugar de las originales.
 - Hacer un análisis de importancia de características y determinar si existe alguna estrategia de eliminación de variables que no sean ponderadas de forma decisiva por los clasificadores, evaluando su impacto en los resultados.
- Elaborar una estrategia de búsqueda de hiperparámetros que optimice los parámetros a buscar, sobre todo para el caso de `LightGBM`. Ejemplos de posibles estrategias podrían ser las librerías Scikit Optimize u Optuna.
- Reproducir este trabajo corrigiendo el bug detectado en el optimizador de hiperparámetros de `LightGBM` que utilizó la métrica *accuracy* en lugar de F1.

Referencias

- [1] BAEZA-YATES, R., GIONIS, A., JUNQUEIRA, F., MURDOCK, V., PLACHOURAS, V., AND SILVESTRI, F. The impact of caching on search engines. In *Proceedings of the 30th annual international ACM SIGIR conference on Research and development in information retrieval* (2007), pp. 183–190.
- [2] CALLAN, J. The lemur project and its clueweb12 dataset. In *Invited talk at the SIGIR 2012 Workshop on Open-Source Information Retrieval* (2012).
- [3] CAMBAZOGLU, B. B., AND BAEZA-YATES, R. Scalability challenges in web search engines. In *Advanced topics in information retrieval*. Springer, 2011, pp. 27–50.
- [4] CHEN, T., HE, T., BENESTY, M., KHOTILOVICH, V., TANG, Y., CHO, H., CHEN, K., MITCHELL, R., CANO, I., ZHOU, T., ET AL. Xgboost: extreme gradient boosting. *R package version 0.4-2* 1, 4 (2015), 1–4.

- [5] DIETTERICH, T. G. Ensemble methods in machine learning. In *International workshop on multiple classifier systems* (2000), Springer, pp. 1–15.
- [6] FAGNI, T., PEREGO, R., SILVESTRI, F., AND ORLANDO, S. Boosting the performance of web search engines: Caching and prefetching query results by exploiting historical usage data. *ACM Transactions on Information Systems (TOIS)* 24, 1 (2006), 51–78.
- [7] FRIEDMAN, J. H. Stochastic gradient boosting. *Computational statistics & data analysis* 38, 4 (2002), 367–378.
- [8] GEURTS, P., ERNST, D., AND WEHENKEL, L. Extremely randomized trees. *Machine learning* 63 (2006), 3–42.
- [9] KE, G., MENG, Q., FINLEY, T., WANG, T., CHEN, W., MA, W., YE, Q., AND LIU, T.-Y. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30 (2017).
- [10] KUCUKYILMAZ, T. Exploiting temporal changes in query submission behavior for improving the search engine result cache performance. *Information Processing & Management* 58, 3 (2021), 102533.
- [11] KUCUKYILMAZ, T., CAMBAZOGLU, B. B., AYKANAT, C., AND BAEZA-YATES, R. A machine learning approach for result caching in web search engines. *Information Processing & Management* 53, 4 (2017), 834–850.
- [12] OZCAN, R., ALTINGOVDE, I. S., AND ULUSOY, Ö. Cost-aware strategies for query result caching in web search engines. *ACM Transactions on the Web (TWEB)* 5, 2 (2011), 1–25.
- [13] PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B., GRISEL, O., BLONDEL, M., PRETTENHOFER, P., WEISS, R., DUBOURG, V., VANDERPLAS, J., PASSOS, A., COURNAPEAU, D., BRUCHER, M., PERROT, M., AND DUCHESNAY, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [14] PODLIPNIG, S., AND BÖSZÖRMENYI, L. A survey of web cache replacement strategies. *ACM Computing Surveys (CSUR)* 35, 4 (2003), 374–398.
- [15] TOLOSA, G. H. *Técnicas de caching de intersecciones en motores de búsqueda*. PhD thesis, Universidad de Buenos Aires. Facultad de Ciencias Exactas y Naturales, 2016.
- [16] TONIN MONZÓN, F., BANCHERO, S., AND TOLOSA, G. H. Árboles de decisión adaptativos en políticas de admisión a caché. In *IV Simposio Argentino de GRANdes DATos (AGRANDA 2018)-JAIIO 47 (CABA, 2018)* (2018).

A. Características del *Dataset*

Tipo	Nombre de la característica	Descripción
Consulta	cantidad_caracteres	Cantidad de caracteres en la consulta
	numero_de_terminos	Cantidad de términos en la consulta
	contiene_URL	Si la consulta contiene una url
	misspelled	Si hay algun error de ortografía
	largo_promedio_terminos	Media de caracteres de los términos de la consulta
	rank_number	posición del resultado elegido en la respuesta devuelta por el motor de búsqueda
	hora_consulta	Hora del día a la que la consulta fue enviada
	frec_consulta_dia	Frecuencia de la consulta en el día
	frec_consulta_hora	Frecuencia de la consulta en la última hora
	frec_consulta_min	Frecuencia de la consulta en el último minuto
Sesión	ctr	Tasa de clicks (media de resultados elegidos para la búsqueda dada)
	ctr_top	Tasa de clicks en el primer resultado
	cant_consultas_realizadas	Cantidad de consultas realizadas en la sesión actual
Índice	min_posting_count	Número de apariciones del término menos frecuente en la colección de referencia
	max_posting_count	Número de apariciones del término más frecuente en la colección de referencia
	avg_posting_count	Número de apariciones promedio de los términos en la colección de referencia
Términos	max_dia_term	Frecuencia del término que más aparece en las consultas del último día
	min_dia_term	Frecuencia del término que menos aparece en las consultas del último día
	avg_dia_term	Frecuencia promedio de los términos en las consultas del último día
	max_hora_term	Frecuencia del término que más aparece en las consultas de la última hora
	min_hora_term	Frecuencia del término que menos aparece en las consultas de la última hora
	avg_hora_term	Frecuencia promedio de los términos en las consultas de la última hora
	max_min_term	Frecuencia del término que más aparece en las consultas del último minuto
	min_min_term	Frecuencia del término que menos aparece en las consultas del último minuto
	avg_min_term	Frecuencia promedio de los términos en las consultas del último minuto

Cuadro 6: Características del *Dataset* al iniciar el trabajo

B. Comparación de métricas calculadas en los experimentos

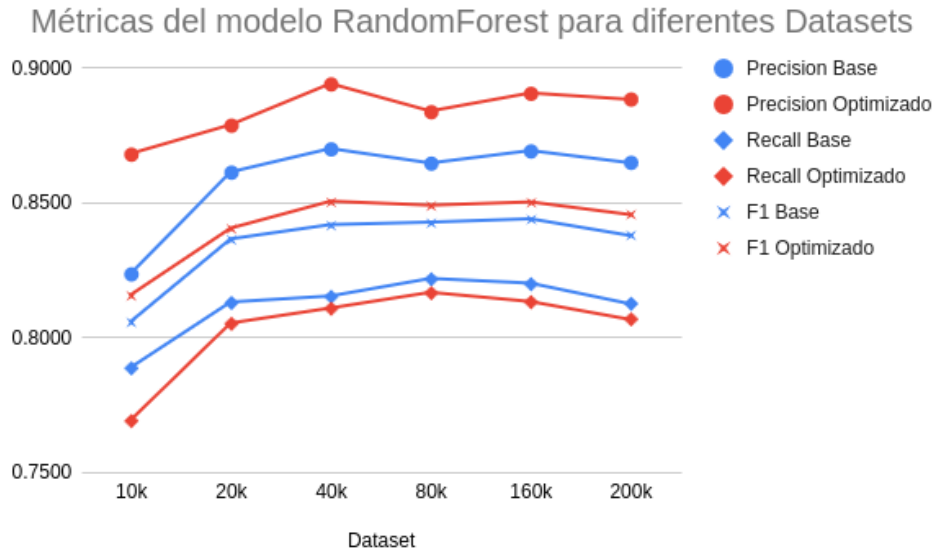


Figura 9: Métricas *precision*, *recall* y *F1* para el modelo **RandomForest** en todos los *datasets*, en versión base y optimizada del clasificador. Gráficamente se observa como *F1* es una media ponderada de las otras dos métricas. La forma de los puntos identifica el modelo, mientras que el color identifica si es el clasificador base o el optimizado. Gráficamente se puede apreciar como la optimización mejoró la métrica *precision* en detrimento de la métrica *recall*, algo que es balanceado oportunamente por la métrica *F1*

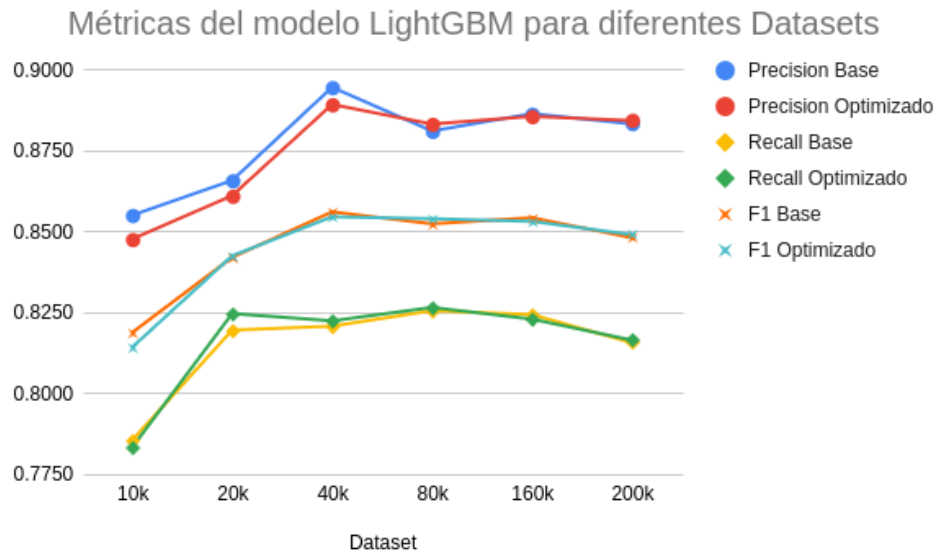


Figura 10: Métricas *precision*, *recall* y *F1* para el modelo **LightGBM** en todos los *datasets*, en versión base y optimizada del clasificador. Gráficamente se observa como *F1* es una media ponderada de las otras dos métricas, ubicándose sus valores entre ambas y evitando picos o valles abruptos, como en el caso del *dataset* 40k.

C. Curvas ROC de los experimentos realizados

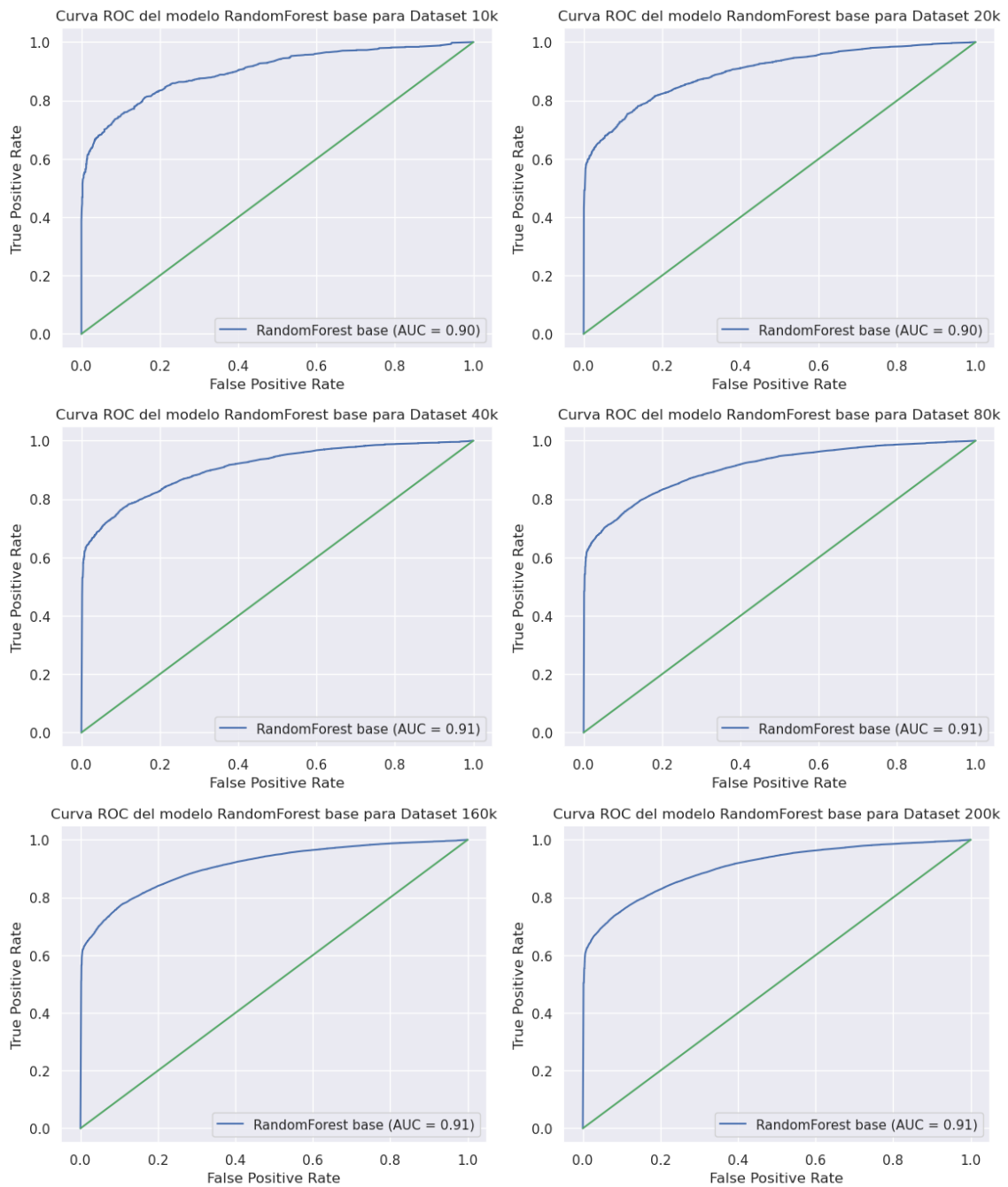


Figura 11: Curvas ROC para el modelo **RandomForest** base en los 6 *datasets*.

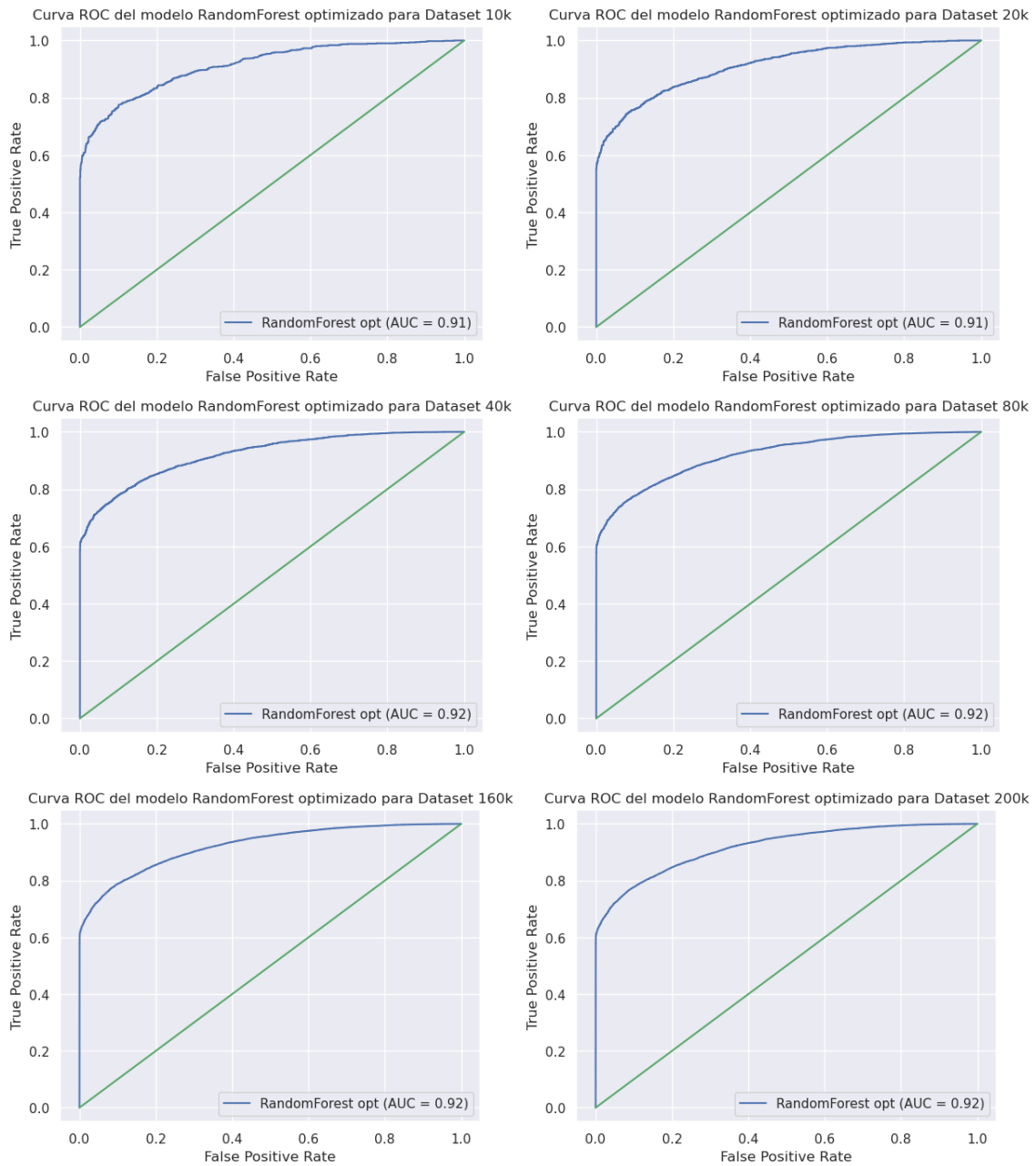


Figura 12: Curvas ROC para el modelo RandomForest optimizado en los 6 datasets.

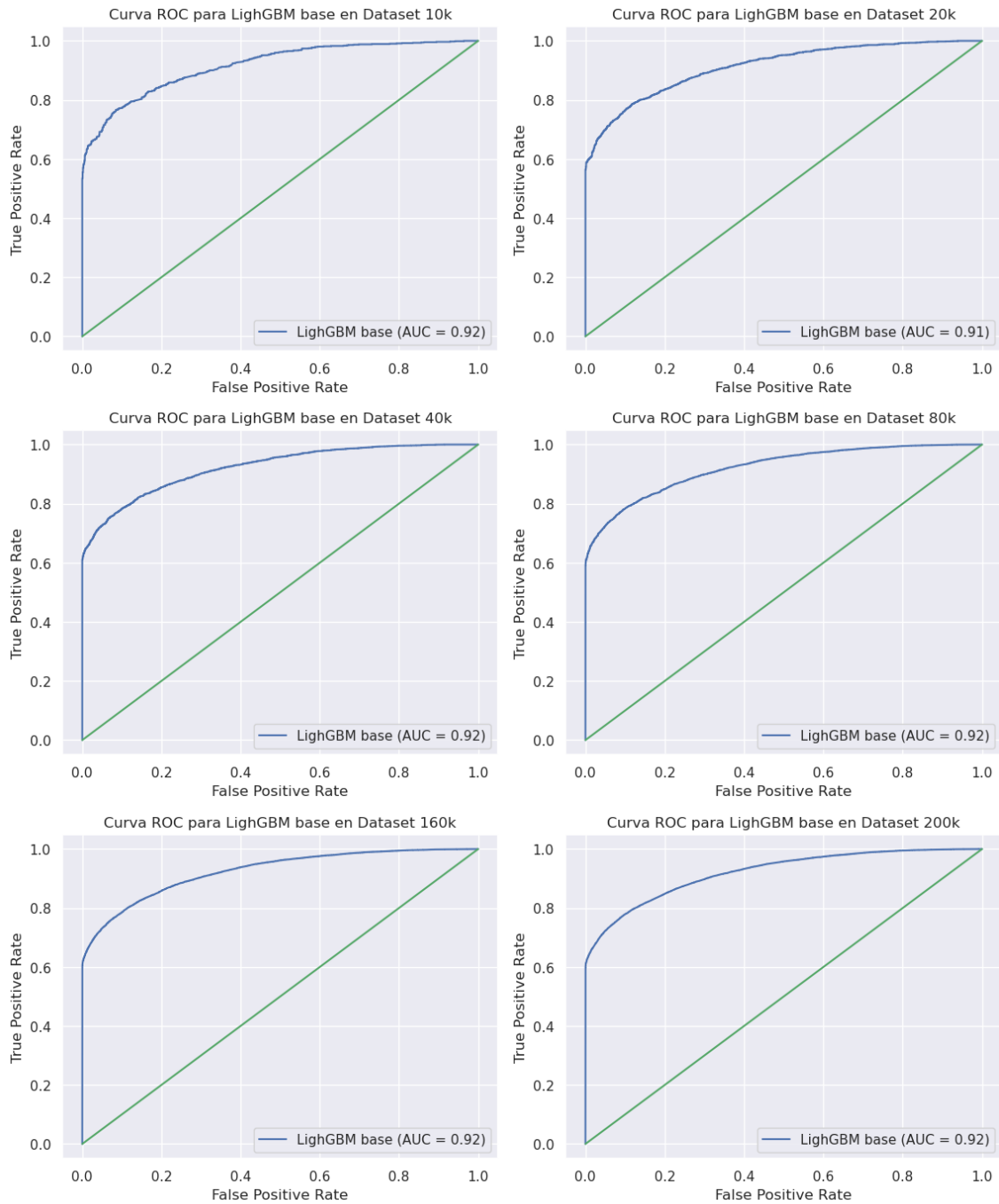


Figura 13: Curvas ROC para el modelo LightGBM base en los 6 datasets.

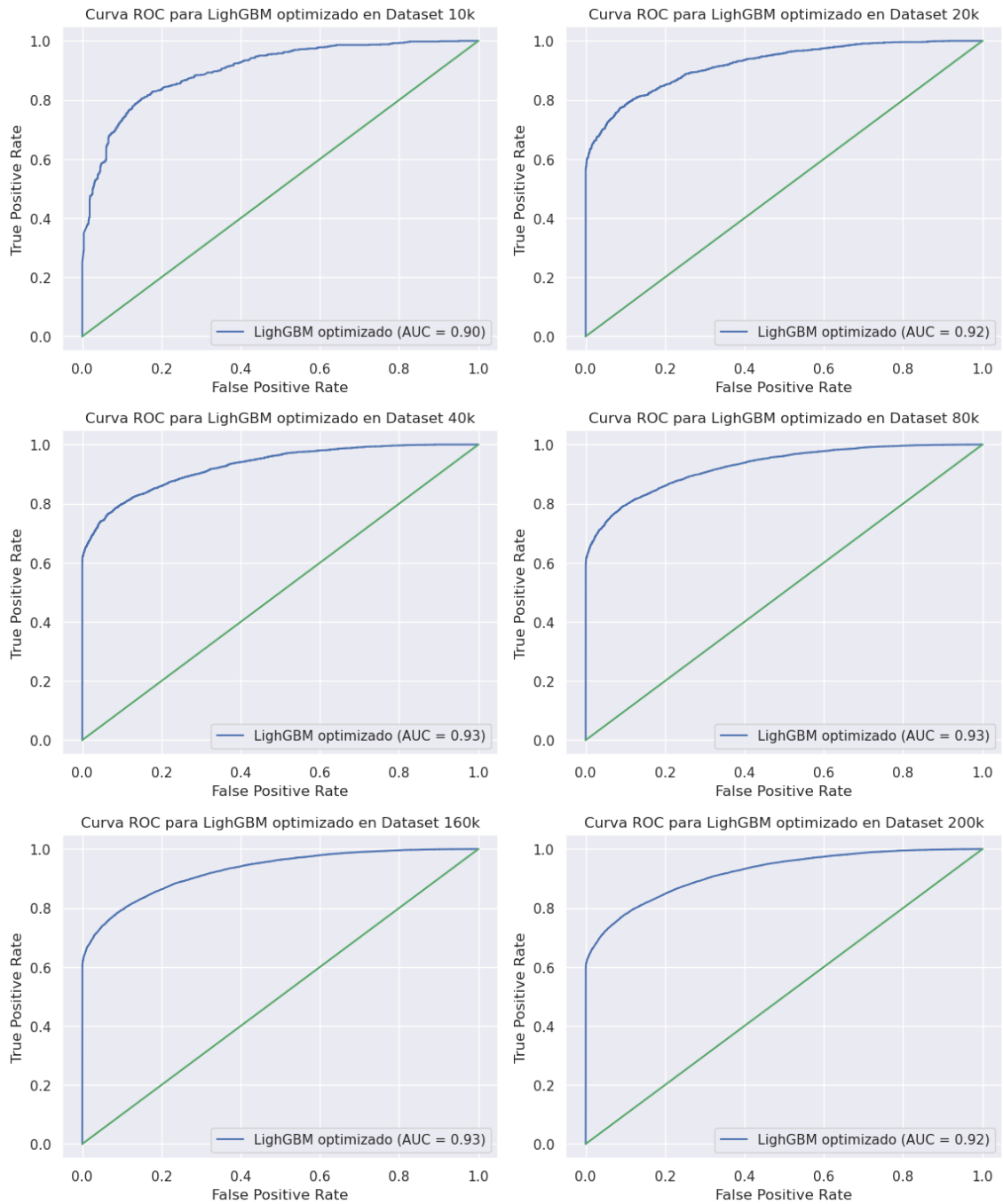


Figura 14: Curvas ROC para el modelo LightGBM optimizado en los 6 datasets.